

O criptossistema RSA - Rivest, Shamir e Adleman



Foto antiga de Adi Shamir, Ron Rivest e Leonard Adleman

Teorema de Fermat

Seja p um número primo:

- i) $\forall a \in \mathbb{Z}, a^p \equiv a \pmod{p}$
- ii) Se $p \nmid a$, então $a^{p-1} \equiv 1 \pmod{p}$

$$a = 7, p = 19$$

$$7^2 = 49 \equiv 11 \pmod{19}$$

$$7^4 \equiv 121 \equiv 7 \pmod{19}$$

$$7^8 \equiv 49 \equiv 11 \pmod{19}$$

$$7^{16} \equiv 121 \equiv 7 \pmod{19}$$

$$a^{p-1} = 7^{18} = 7^{16} \times 7^2 \equiv 7 \times 11 \equiv 1 \pmod{19}$$

$$p = 5, a = 3, 3^5 = 243 \equiv 3 \pmod{5}$$

$$p = 5, a = 10, 10^5 = 100000 \pmod{5} \equiv 0 \pmod{5}$$

Demonstração

-ii) Supondo que $p \nmid a$. Tem-se que os inteiros $\{1a, 2a, \dots, (p-1)a\}$ formam o conjunto dos resíduos módulo p . Isto pode ser mostrado da maneira a seguir: “Pegando” dois resíduos ia e ja , os mesmos têm que estar na mesma classe de resíduos, isto é: $ia \equiv ja \pmod{p} \Rightarrow p \mid (i-j)a$. Como $p \nmid a \Rightarrow p \mid (i-j)$. Como $i < p$ e $j < p \Rightarrow i=j$. Logo, os inteiros $1, 2, 3, \dots, p-1$ é um rearranjo (recomposição) de $a, 2a, 3a, \dots, (p-1)a$ módulo $p \Rightarrow a^{p-1}(p-1)! \equiv (p-1)! \pmod{p} \Rightarrow p \mid ((p-1)! (a^{p-1}-1))$. Como $p \nmid (p-1)! \Rightarrow p \mid (a^{p-1}-1)$, com requerido ($a^{p-1} \equiv 1 \pmod{p}$).

-i) Multiplicando ambos os lados da congruência $a^{p-1} \equiv 1 \pmod{p}$ por $a \Rightarrow a \cdot a^{p-1} \equiv a \cdot 1 \pmod{p} \Rightarrow a^p \equiv a \pmod{p}$, quando $p \nmid a$.

-Se $p \mid a$, a congruência $a^p \equiv a \pmod{p}$ é trivial, pois ambos os lados são $\equiv 0 \pmod{p}$ (a é múltiplo de p)

Função phi de Euler

$\phi(n)$ é

Número de Inteiros positivos menores que **n** e relativamente primos a **n**

$$\phi(m) = \prod_{i=1}^m (p_i^{e_i} - p_i^{e_i-1})$$

Se p e q são primos então $\phi(p) = p-1$ e $\phi(q) = q-1$

$$\phi(pq) = \phi(p)\phi(q) = (p-1)(q-1)$$

$$\phi(21) = 12 = \phi(3)\phi(7) = 2 \times 6 = (3-1)(7-1)$$

onde os 12 inteiros são {1,2,4,5,8,10,11,13,16,17,19,20}

Teorema de Euler

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

Com **a** e **n** relativamente primos

$$a = 3; n = 10; \phi(10) = 4; 3^4 = 81 \equiv 1 \pmod{10}$$

$$a = 2; n = 11; \phi(11) = 10; 2^{10} = 1024 \equiv 1 \pmod{11}$$

Aspectos Computacionais - Exponenciação

$$[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$$

Seja $m = b_k b_{k-1} \dots b_0$

$$m = \sum_{b_i \neq 0} 2^i$$

$$a^m = a^{\sum_{b_i \neq 0} 2^i} = \prod_{b_i \neq 0} a^{2^i}$$

$$a^m \bmod n = \left[\prod_{b_i \neq 0} a^{2^i} \right] \bmod n = \prod_{b_i \neq 0} a^{2^i} \bmod n$$

$$d = a^m \bmod n$$

```
d = 1
para i = k passo -1 até 0 faça
    d = (d x d) mod n
    se bi = 1 então
        d = (d x a) mod n
    fim se
fim para
retorna d
```

Criptografia de chave pública

- **Alguns algoritmos de chave pública**
 - RSA : Utiliza teoria dos números. Segurança: dificuldade de fatorar números inteiros grandes.
 - McEliece: Utiliza teoria algébrica dos códigos lineares. Segurança: Dificuldade de decodificar códigos de blocos lineares.
 - ElGamal: Utiliza corpos finitos. Segurança: Dificuldade do problema do logaritmo discreto para corpos finitos.
 - Elliptic Curve: Utiliza Curvas elípticas.

Simétrica x Assimétrica

- | | |
|---|--|
| <ul style="list-style-type: none">• Rápida | <ul style="list-style-type: none">• Lenta |
| <ul style="list-style-type: none">• Gerência e distribuição das chaves é complexa | <ul style="list-style-type: none">• Gerência e distribuição das chaves é simples |
| <ul style="list-style-type: none">• Não oferece assinatura digital | <ul style="list-style-type: none">• Oferece assinatura digital |

Sistema Híbrido - exemplo

- Emissor cria uma chave simétrica;
- Emissor criptografa a mensagem com a chave simétrica criada;
- Emissor criptografa a chave simétrica com a chave pública do receptor;
- Emissor envia a mensagem e a chave criptografada
- Receptor descriptografa a chave utilizando a sua chave secreta;
- Receptor descriptografa a mensagem utilizando a chave simétrica que descriptografou anteriormente.

Protocolos que empregam Sistemas Híbridos

- IPSEC - Padrão desenvolvido para o IPv6;
- SSL e TLS - Utilizado nos protocolos NTTP, HTTP, SMTP e Telnet;
- PGP - Criptografia de e-mail pessoal;
- S/MINE - Criptografia de e-mail corporativo;
- SET - Transações financeiras;
- X.509 - Relacionamento entre as autoridades de certificação.

Criptografia de chave pública

- É assimétrica porque utiliza um par de chave – Uma para encriptar e a outra para decriptar a mensagem;
- Desenvolvida para resolver duas questões:
 - Distribuição de chaves
 - Assinaturas digitais
- Invenção pública por Whitfield Diffie & Martin Hellman na Universidade de Stanford em 1976

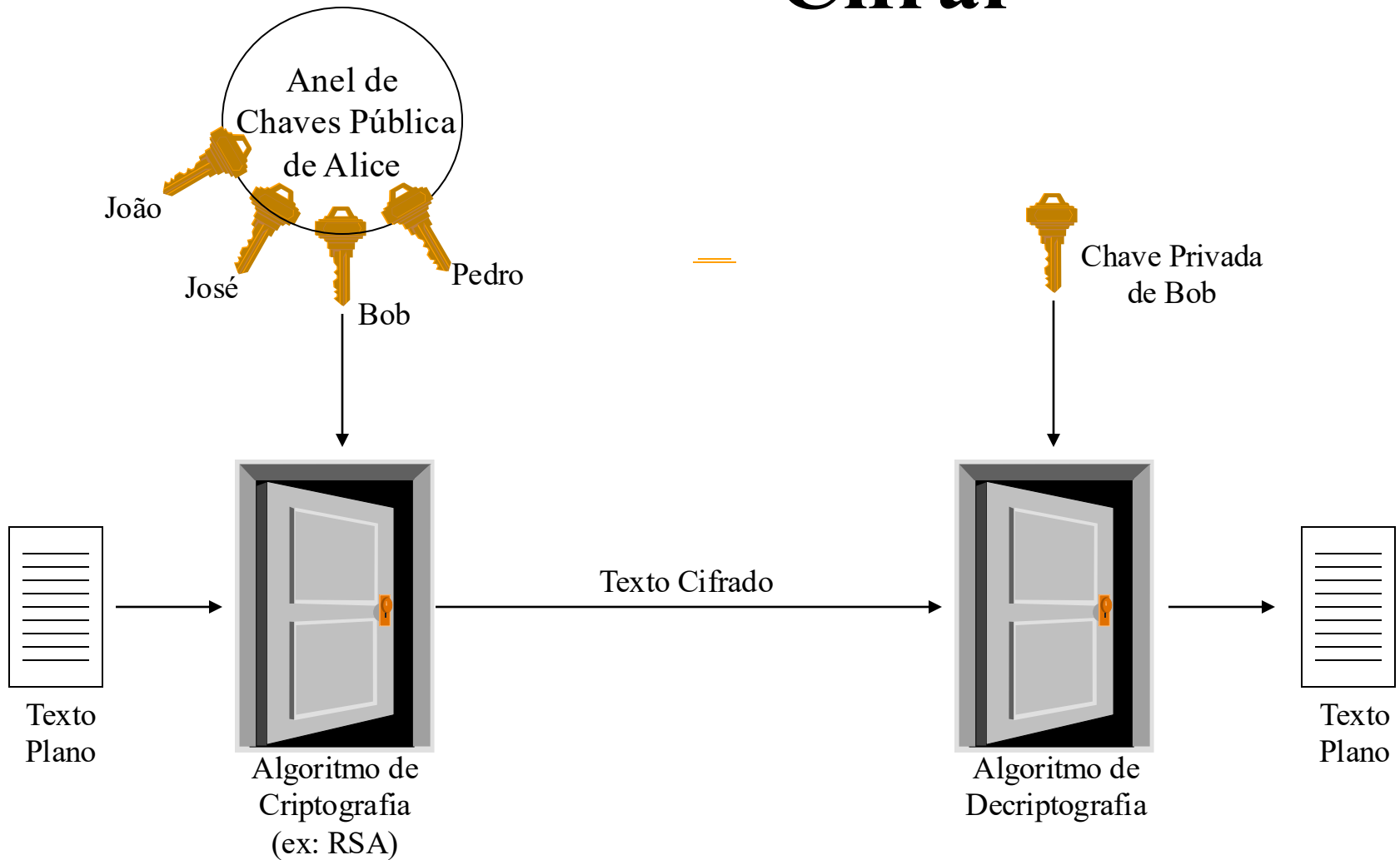
Criptografia de chave pública

- É o avanço mais significativo em 3000 anos de criptografia;
- Utiliza-se duas chaves que devem ter as características:
 - Computacionalmente intratável determinar a chave de decifragem, conhecendo apenas o algoritmo e a chave utilizada na cifragem;
 - Computacionalmente “fácil” cifrar/decifrar mensagens quando a chave correta é conhecida;
 - Alguns algoritmos permitem que as duas chaves possam ser utilizadas para cifrar ou decifrar.

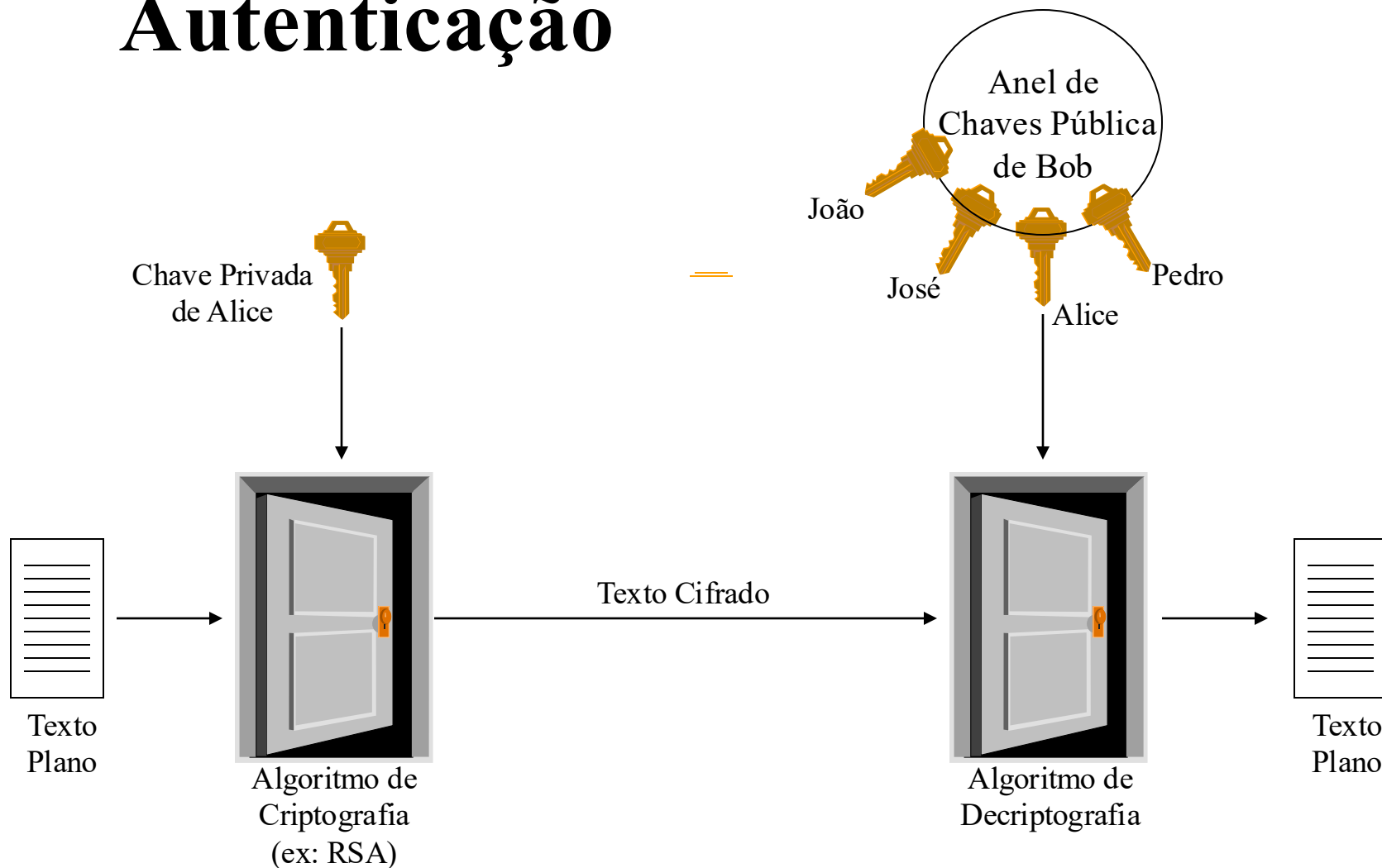
Criptografia de chave pública

- Funções unidirecionais
 - Dado x é de fácil complexidade calcular $f(x)$, mas é intratável calcular x através de $f(x)$, ou seja, “fácil” de se calcular $f(x)$ e difícil de se calcular $f^{-1}(x)$.
- Os algoritmos de chave pública utilizam funções unidirecionais com segredo.

Cifrar



Autenticação



Criptografia de chave pública - O RSA

- Desenvolvido por Ron Rivest, Adi Shamir e Len Adleman em 1977;
- Algoritmo de chave pública mais conhecido e utilizado;
- Baseado em exponenciações de inteiros utilizando aritmética modular:
 - Exponenciação leva $O((\log n)^3)$ operações (fácil cifrar/decifrar).
- Utiliza inteiros muito grandes (por exemplo: 2048 bits);
- Segurança: Dificuldade de fatorar n^{os} inteiros grandes
 - Fatoração leva $O(e^{\log n \log n \log n})$ operações (difícil obter a chave privada a partir da chave pública)

Algoritmo RSA

Geração da Chave

Selecione p, q	p e q primos
Calcular $n = p \times q$	
Calcular $\phi(n) = (p-1)(q-1)$	
Selecionar e inteiro	$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$
Calcular d	$d = e^{-1} \bmod \phi(n)$
Chave Pública	$KU = \{e, n\}$
Chave Privada	$KR = \{d, n\}$

Cifrar

Texto Plano:	$M < n$
Texto Cifrado:	$C = M^e \bmod n$

Decifrar

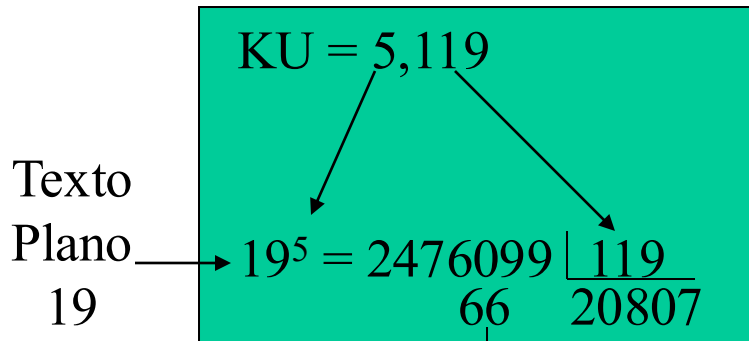
Texto Cifrado:	C
Texto Claro:	$M = C^d \bmod n$

Exemplo - RSA

- Selecionar dois números primos: $p = 7$ e $q = 17$
- Calcular $n = p \cdot q = 7 \times 17 = 119$
- Calcular $\phi(n) = (p-1) \cdot (q-1) = 96$
- Selecionar e tal que e é relativamente primo a $\phi(n)$ e menor que $\phi(n)$; $e = 5$
- Determinar d tal que $d \cdot e = 1 \pmod{96}$ e $d < 96$;
 $d = 77$, pois $77 \times 5 = 385 = 4 \times 96 + 1$
- $K_p = \{5, 119\}$ e $K_s = \{77, 119\}$

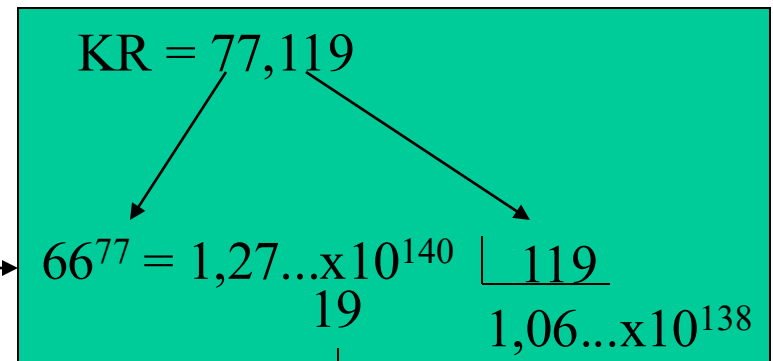
Continuação do Exemplo

Cifrar



Texto Cifrado 66

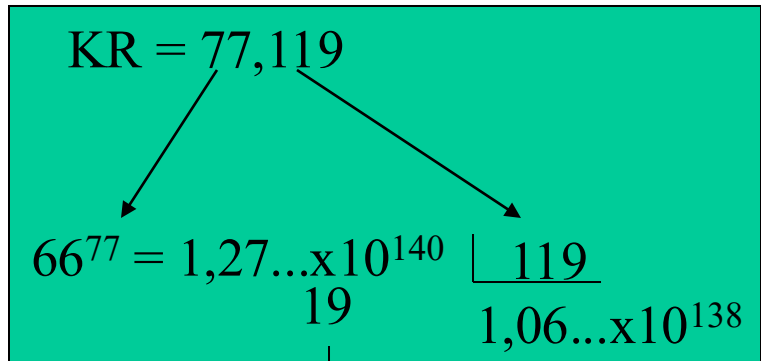
Decifrar



Texto Plano 19

Exercício

Decriptar



Usar o algoritmo para calcular
 $d = a^b \bmod n$

$$d = 66^{77} \bmod 119 = ?$$

Porque é que o RSA funciona

- Teorema de Euler: $a^{\phi(n)} \equiv 1 \pmod n$
- No RSA temos:
 - $n=p.q$ e $\phi(n)=(p-1).(q-1)$
 - e & d são inversos mod $\phi(n)$: $e.d \equiv 1 \pmod{\phi(n)}$
 - portanto: $e.d = 1+k.\phi(n)$
 - $C^d = (M^e)^d = M^{1+k.\phi(n)} = M^1.(M^{\phi(n)})^k = M^1.(1)^k = M^1 = M$

Criptografia de chave pública - RSA

- Considerações
 - A dificuldade de se quebrar o RSA baseia-se na intratabilidade de fatorar n na ordem de 618 dígitos decimais (2048 bits) ou de determinar $\varphi(n)$ ou d , *que* são problemas computacionalmente equivalentes.
 - Os números p e q devem diferir em alguns bits de comprimento (caso contrário p e q ficarão muito próximos da RAIZ(n)).
 - Os números $p-1$ e $q-1$ devem conter fatores primos grandes.
 - O $\text{mdc}(p-1, q-1)$ deve ser pequeno.
 - Obs.: 4096 bits $\cong$ 1234 dígitos decimais, 1024 bits $\cong$ 310 dígitos decimais.

Geração da chave do RSA

- Os utilizadores do RSA têm que:
 - determinar dois números primos ao acaso: **p** e **q**;
 - seleccionar **e** ou **d** e calcular o outro.
- Os primos p e q não podem ser determinados facilmente a partir de n ($n=p.q$):
 - têm que ser suficientemente grandes;
 - não há métodos directos para determinar primos muito grandes;
 - tipicamente usam números aleatórios e testes de “primalidade”. Esses testes não garantem que o número seja primo.

Geração da chave do RSA

- Algoritmo de Miller-Rabin
 - Escolhe **n** (ímpar) e faz-se: **$n-1 = 2^k m$** , com **m** ímpar
 - Escolhe um inteiro **a** (aleatório), **$1 \leq a \leq n-1$**
 - Calcula: **$b = a^m \bmod n$**
 - **se $b \equiv 1 \pmod{n}$**
 - então escreva n, “é primo” e sair**
 - para **i=0** até **k-1** faça
 - **se $b \equiv -1 \pmod{n}$**
 - então escreva n, “é primo” e sair**
 - else $b = b^2 \bmod n$**
 - escreva **n “é composto”**

Geração da chave do RSA

- Considerações:
 - a probabilidade de um número ímpar aleatório p ser primo é:
 - $P(p \text{ ser primo}) \cong \frac{2}{\ln(p)}$
 - Exemplo: para o RSA com módulo $n = 1024$ -bit, os primos p e q devem ter comprimento de cerca de 512 bits, ou seja, p e $q \cong 2^{512}$. Logo:
 - $P(p \text{ ser primo}) \cong \frac{2}{\ln(2^{512})} \cong \frac{2}{512\ln(2)} \cong \frac{2}{512 \times 0,693} \cong \frac{1}{177}$
 - Exercício:
 - Calcular $P(p)$ para o RSA com módulo $n = 2048$ bits

Geração da chave do RSA

- Considerações:
 - O número de execuções no teste de primalidade do algoritmo de Miller–Rabin, para uma probabilidade de erro inferior a 2^{-80} , é:

Comprimento em bits de p	Número de execuções (s=segurança)
250	11
300	9
400	6
500	5
600	3

Geração da chave do RSA

- Algoritmo de Miller-Rabin - Exemplo
 - Testar se o número 91 é primo;
 - Realizar 4 testes:
 - Primeiro teste $\implies a = 12$;
 - Segundo teste $\implies a = 17$;
 - Terceiro teste $\implies a = 38$;
 - Quarto teste $\implies a = 39$;
 - Mostre o resultado de cada teste.